Spark Fuse Cloud GPU Bridge — User Manual

Version 0.3.0 · July 2026

The Spark Fuse Cloud GPU Bridge is a ComfyUI extension that renders your current workflow on a Spark Fuse cloud GPU and brings the finished images straight back into ComfyUI. You keep authoring locally; the cloud does the heavy lifting.

---

1. When the bridge makes sense

Every cloud render pays a per-job startup overhead (provisioning, image pull on a cold node, model load) before the first sampling step. A fast local GPU will beat the cloud on models that fit in your VRAM. The bridge earns its keep when:

- **Your models exceed your local VRAM.** Run checkpoints and text encoders your card cannot hold, on a GPU class you do not own.
- **You have long batch queues.** Queue the work and keep using (or switch off) your workstation; the warm-instance queue amortises the startup cost across many jobs.
- **You have no GPU, or a small one.** Author workflows anywhere ComfyUI runs and let the cloud do all the lifting.

2. How it works

The bridge is an extension, not a graph node. Clicking **Render on Spark Fuse** ships your current graph (in ComfyUI API format) to Spark Fuse, which runs it on the published ComfyUI Docker image. Your model library lives on ShareSync and is mounted read-only at `/assets` inside the cloud container, cached on the compute node across jobs. Only the small `workflow.json` travels per render; models are staged once and reused.

If a workflow references models that are not on ShareSync yet, the bridge detects that **before** submitting and walks you through syncing them (section 7).

3. Requirements

- A local ComfyUI install on Python 3.12 or newer.
- A Spark Fuse account with API credentials.
- The `spark-fuse-messenger` Python package, version 0.5.0 or newer, installed into the same Python that runs your ComfyUI (the install steps cover this).

4. Install

1. Install the extension into ComfyUI's `custom_nodes` folder.

   Via ComfyUI-Manager (easiest): search "Spark Fuse Cloud GPU Bridge" and install.

   Or clone it directly so it sits at `ComfyUI/custom_nodes/spark-fuse-comfyui-node`:

   ```
   git clone https://github.com/VFXGuru/spark-fuse-comfyui-node
   ```

2. Install the messenger client into the Python that runs your ComfyUI.

   Standard install (a venv or system Python):

   ```
   pip install -r requirements.txt
   ```

   Portable or desktop ComfyUI (embedded Python): the portable build ships its own Python in a `python_embeded` folder. From that folder:

   ```
   .\python.exe -m pip install hatchling
   .\python.exe -m pip install --no-build-isolation -r "..\ComfyUI\custom_nodes\spark-fu
   ```

   Confirm with `.\python.exe -c "import spark_fuse; print('spark_fuse OK')"`.

3. Restart ComfyUI. A ⚡ **Spark Fuse** button appears at the top right.

5. Configure

Click ⚡ **Spark Fuse**, open **Credentials**, and set your host, email and password (or provide `SPARK_HOST`, `SPARK_EMAIL` and `SPARK_PASSWORD` as environment variables). Then:

- **Assets ShareSync path**: the ShareSync folder that holds your model subfolders, for example `/comfy-flux2-klein/models`. The cloud mounts this at `/assets`. The panel previews the resolved path live as you type and validates the leading slash before submit.
- **GPU**: pick an instance type; the hourly rate shows beside it. The list shows GPU instances only, grouped by family and size.
- **Image affinity**: `required` reports whether repeated runs hit the image cache; `preferred` does the same placement silently.
- **Batch count**: how many images one job renders (section 6.1).

Click **Save settings**. Settings persist in `spark_fuse_settings.json` under ComfyUI's user directory, not inside the extension folder, so they survive a node update, uninstall or reinstall; prefer environment variables on shared machines.

6. Rendering

1. Build or open a workflow as usual, ending in a Save Image node.
2. Click ⚡ **Spark Fuse**, choose a GPU, and click **Render on Spark Fuse**.
3. The bridge first checks your models against ShareSync (section 7), then submits. Progress streams into the panel; the finished image appears in the panel and lands in ComfyUI's output folder under a fresh sequential name.

6.1 Batch render

Set **Batch count** to render several images from one job. The job pays the cold start and loads the model once, then renders that many images in sequence with fresh seeds. VRAM use stays at one image's worth, and every image after the first costs only its sampling time. All images download into ComfyUI's output folder.

Each seed's **control_after_generate** setting (`fixed` , `increment` , `decrement` , `randomize`) is honoured on both single renders and queued items, matching what ComfyUI does locally: `fixed` keeps the same seed; the other three advance it before the next render or queue addition.

6.2 Render queue

The queue runs several different workflows back to back on one pre-warmed instance. Several workflows share a single job and a single ComfyUI process, so only the first pays the model load; each one after that costs roughly its own sampling time. Long queues are split into batches of up to 10 workflows per job, every batch still routed onto the same prepared instance.

1. Open a workflow, set its batch count, click **Add to queue**. Repeat per workflow; each item snapshots the graph as it is when added.
2. Click **Run queue**. The bridge checks the models of **all** queued workflows first (one consolidated consent if anything needs syncing), then prepares the instance and runs each batch in turn. Per-workflow status (queued, running, succeeded, failed) updates live as each workflow in a batch executes; images download once that batch's job finishes. A workflow that fails does not stop the rest.
3. **Cancel queue (finishes current batch)** lets the batch currently running finish and download normally — nothing already rendered is lost — then stops before submitting any further batches, and releases the instance.

The prepared instance is billed for the whole session including the gaps between batches, so run the queue back to back rather than leaving it idle.

7. Pre-render model sync

Before any submit, the bridge scans the workflow for the models it references (checkpoints, LoRAs, VAEs, text encoders, ControlNets, CLIP Vision, upscale models, GGUF files) and checks each against ShareSync by **filename and exact size**. Two rules govern everything:

- **Nothing uploads without your explicit consent.**
- **Nothing submits while a referenced model is missing.**

7.1 What the check can find

Everything present. The render submits immediately; the check adds well under a second.

Missing on ShareSync, present locally. A consent list appears in the panel showing each model's name, folder and size, with three buttons:

1. **Upload N model(s), then render.** Blocks this render until the sync completes, then submits. Progress shows in the status line and the badge.
2. **Upload for next time (render cancelled).** Nothing is submitted this run; the upload continues in the background so the model is on ShareSync for your next render.
3. **Cancel.** No upload, no render.

Missing everywhere. The model is neither local nor on ShareSync; no upload can fix that. The bridge names the model and stops. Nothing runs.

Already uploading. A model from an earlier "upload for next time" is still transferring. The render is held off until it finishes; watch the badge.

7.2 The upload badge

Background uploads get a persistent badge under the ⚡ button showing the current file, percent, bytes transferred and a rough time remaining. It stays visible when the panel is closed and re-attaches after a browser tab reload, because the transfer runs inside the ComfyUI server process, not the browser. The ✕ on the badge cancels the upload.

Closing ComfyUI itself stops the transfer. That is safe: an interrupted upload is detected by the size check and simply re-offered on the next render.

7.3 Where uploads go

Each model uploads to:

```
{assets ShareSync path}/{model folder}/{name}
```

preserving local subfolders. For example, with the default assets path, `FLUX2\flux2-dev.safetensors` in your local `diffusion_models` folder lands at `/comfy-flux2-klein/models/diffusion_models/FLUX2/flux2-dev.safetensors`. The cloud library keeps mirroring your local model layout, which is exactly what the cloud ComfyUI expects.

7.4 The upload guard

Files larger than a configurable guard (default **50 GB**) are never uploaded from the node. They appear in the consent list with a note to stage them via the ShareSync desktop app instead, or to raise the guard if

you have the bandwidth.

This is a practicality limit, not a server one: ShareSync accepts files up to 2 TB, but transfers are not resumable, so an interrupted huge upload restarts from zero. The guard is editable in the panel under **Credentials** as "Upload guard (GB, max single-file auto-upload)", and lives in `spark_fuse_settings.json` under ComfyUI's user directory as `"upload_guard_gb"` (`0` disables it).

7.5 Overwrites

If ShareSync holds a file with the same name but a different size, the bridge treats it as out of date: the consent list flags it ("will overwrite the cloud copy") and an approved upload replaces it.

8. Settings reference

`spark_fuse_settings.json` (under ComfyUI's user directory, created by **Save settings**; survives node updates, uninstall and reinstall):

Key	Default	Meaning
`host`	production API host	Spark Fuse API endpoint
`email` / `password`	empty	Credentials (or use `SPARK_EMAIL` / `SPARK_PASSWORD`)
`instance_type`	`g7e.2xlarge`	GPU SKU
`assets_share_sync_path`	`/comfy-flux2-klein/models`	ShareSync folder mounted at `/assets`
`assets_share_sync_space_name`	empty	ShareSync Project holding the assets path (empty = Personal space)
`image_affinity`	`required`	Cached-image placement reporting
`batch_count`	`1`	Images per job (1 to 100)
`upload_guard_gb`	`50`	Model-sync upload guard in GB; `0` disables

9. Troubleshooting

- **"Spark Fuse credentials are missing."** Set them in the panel's Credentials section, or export `SPARK_HOST`, `SPARK_EMAIL`, `SPARK_PASSWORD`.
- **"Model(s) not found locally or on ShareSync."** The workflow references a file that exists nowhere the bridge can see. Fix the loader's selection, or put the file into your local models folder and render again (the bridge will then offer to upload it).
- **"exceeds the configured upload guard."** Stage the file with the ShareSync desktop app, or raise `upload_guard_gb` in `spark_fuse_settings.json`.
- **"Model upload failed."** The transfer hit an error after its automatic retry. Render again: the size check re-detects the file and re-offers the upload from the start.

- **A job fails with "not found on the cloud."** The model is missing under `/assets` at the expected path. Check that your ShareSync assets folder mirrors your local model folders, or let the model sync stage it for you.
- **No warm-pool capacity (queue).** Governed by `session_affinity`, a setting distinct from **Image affinity** above and not currently exposed in the panel (set it by hand in `spark_fuse_settings.json` if needed). It defaults to `preferred`, which falls back to independent submits; `required` aborts the queue instead. Retry later.
- **The ⚡ button or panel does not appear.** Hard-refresh the browser tab (Ctrl+F5) and check the ComfyUI log and browser console.

10. Licence